Vulnerability Benchmark System — Complete Setup Guide

Complete setup guide (frontend + backend + PostgreSQL).

---

1) Items Included

1. Frontend ZIP (the `frontend` project)
2. Backend folder (`backend`) or backend source code
3. SQL schema file: `cve_queries.sql`
4. Core dump link (Google Drive) for CVE data

Use this link for the dump and query file link:

- `https://drive.google.com/drive/folders/1N5EaPb7PlsMACFoMomphd0LEBDNJYU6r`

---

2) Prerequisites (install once)

Required

- Python 3.11+ (recommended: 3.12)
- PostgreSQL (running locally)
- Node/Bun/Yarn for frontend (Bun recommended)

Recommended tools

- pgAdmin (optional, for DB inspection)
- Git

---

3) Project folder layout expected

After extraction, expected structure:

- Backend (vibe coded. Change the format according to your needs later.)
- frontend

- Cve_queries.sql (download from Drive), e.g. `cve_queries.sql`
- Core dump file (download from Drive), e.g. `cve_record.sql`

---

4) Backend & Frontend setup (FastAPI)

Backend

Step 4.1.1 — Create Python virtual environment

From project root or in the backend folder:

python3 -m venv .venv
source .venv/bin/activate

Step 4.1.2 — Install backend dependencies

cd backend
pip install -r requirements.txt

Step 4.1.3 — Configure backend environment

Copy env template:

cp .env.example .env

Step 4.1.4 — Set Env Variable to connect the database after setting up the db.

Open backend/.env and set values:

- `DATABASE_URL=postgresql://<db_user>:<db_pass>@localhost:<db_port>/<db_name>`

Example:

DATABASE_URL=postgresql://postgresuser:password123@localhost:8003/vulnmark

FrontEnd

Step 4.2.1 — Configure Frontend

- Cd frontend/

- Run:
  - **bun i / npm i / yarn add**
- Run:
  - **Bun run dev / npm run dev / yarn run dev**

---

5) Database setup (PostgreSQL)

Step 5.1 — Install PgAdmin4 and Create database

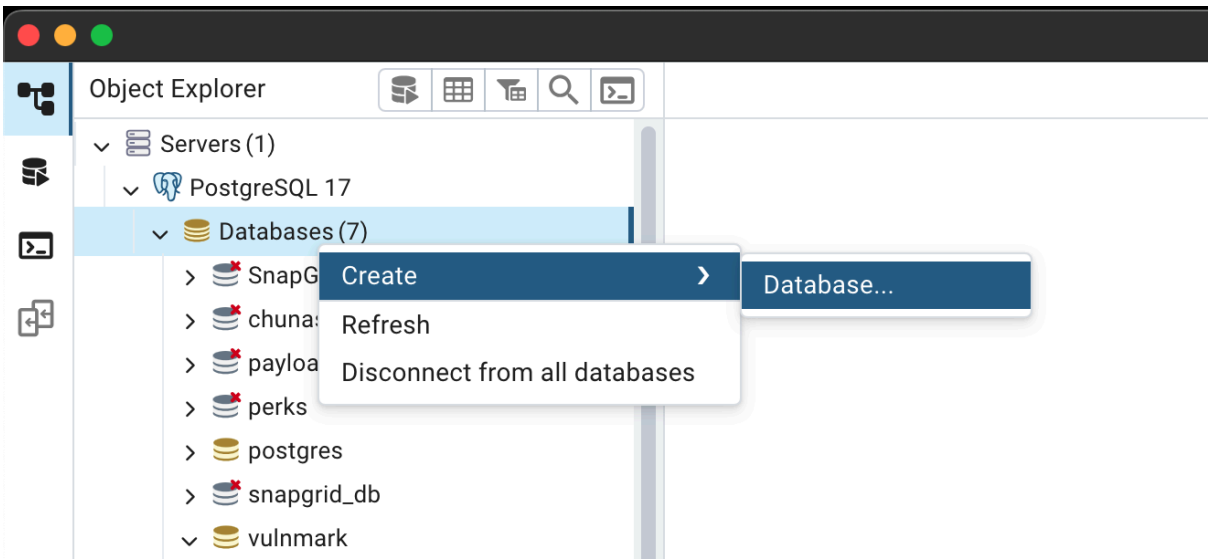
For reference:

- PostgreSQL Setup Guide for Windows
- PostgreSQL Setup Guide for Mac

After Installation:

Set your credentials (username and password) for pgAdmin. The user is ‘postgres’ in most scenarios so you can keep the same. Make sure to remember the password or note it down somewhere.

Create DB:

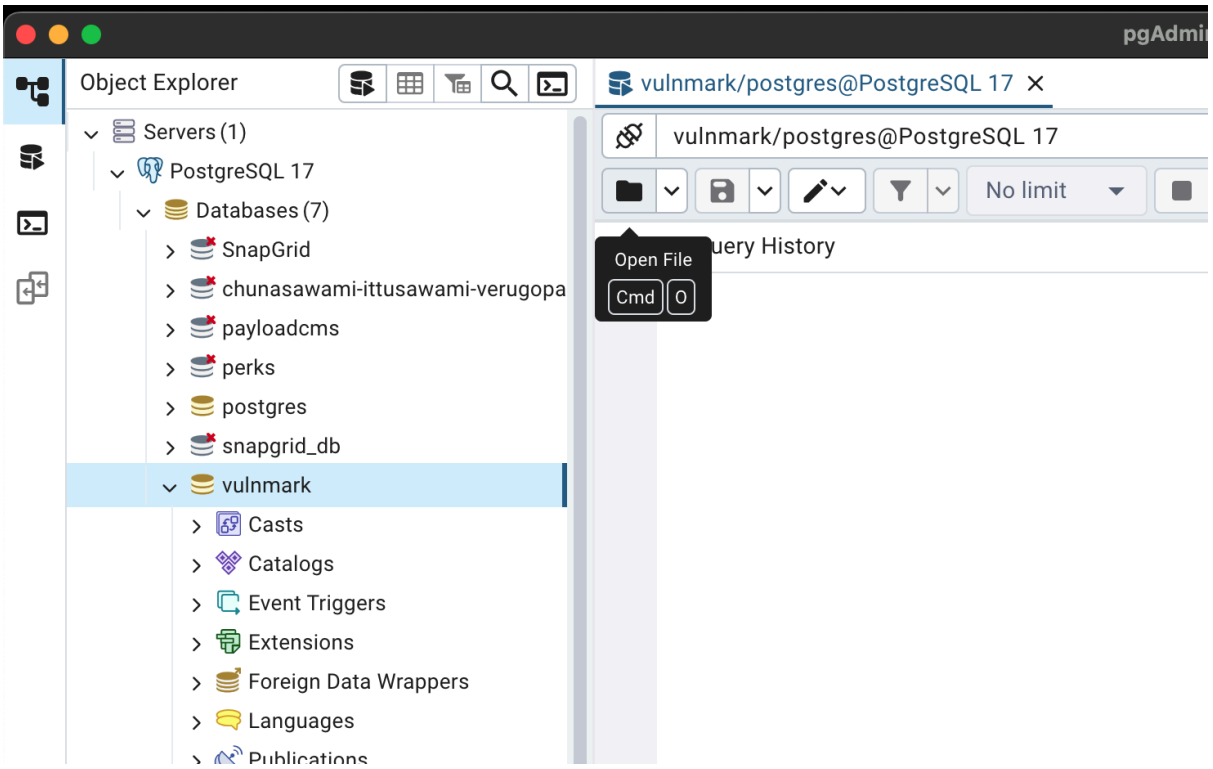


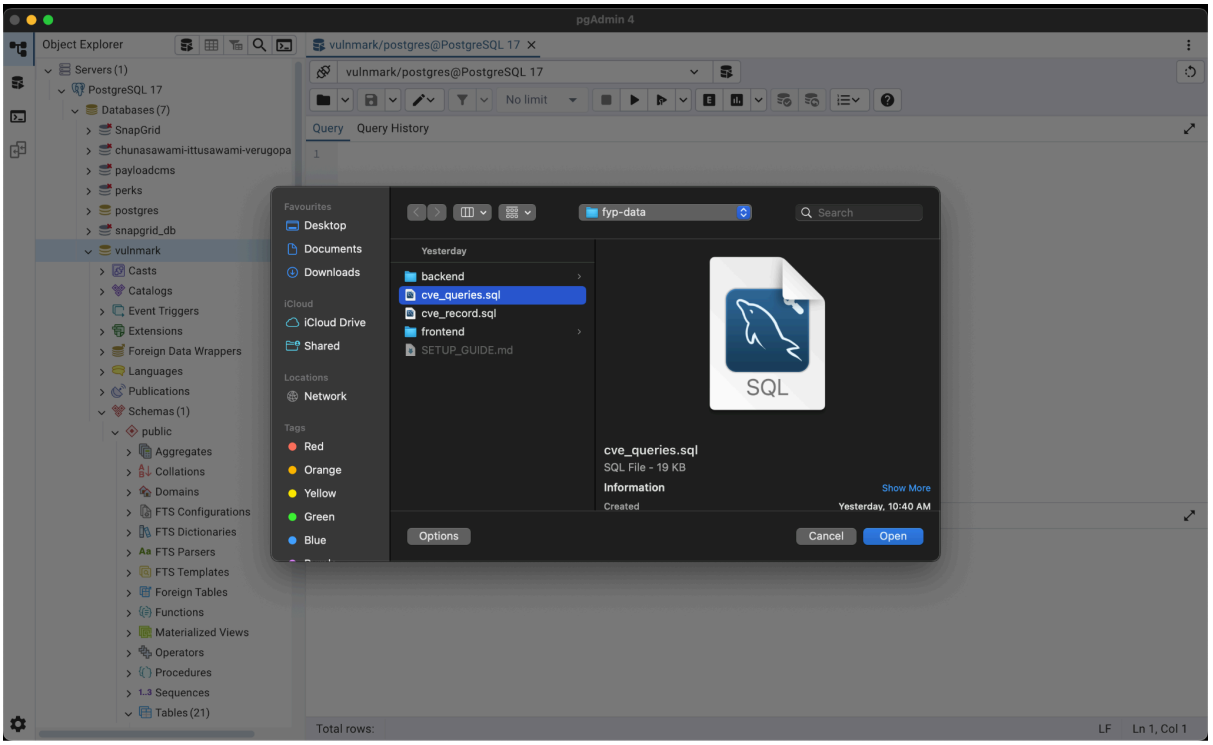
Import the query file:

Step 5.2 — Apply schema and materialized views

From project root (where `cve_queries.sql` exists):

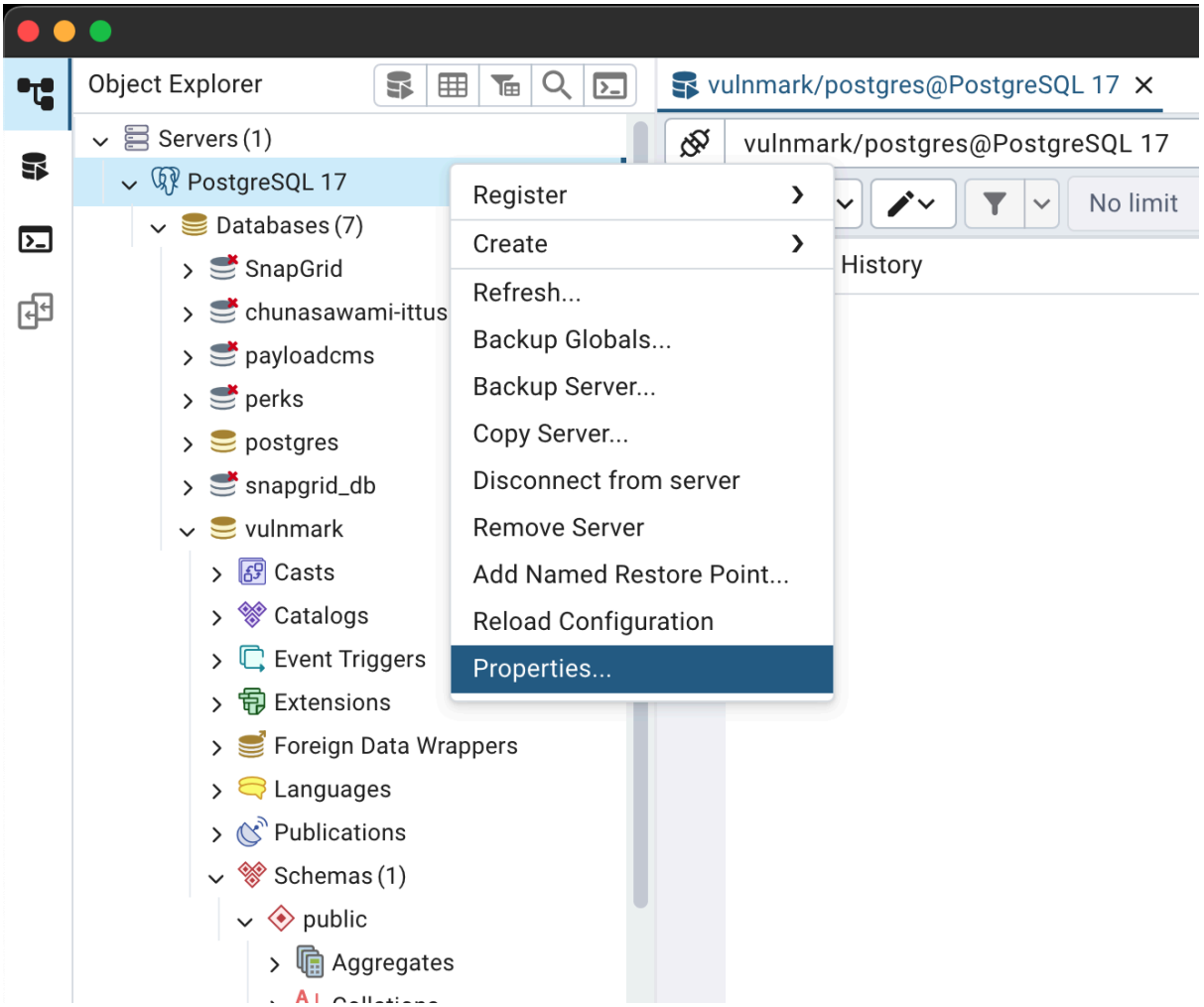
PGPASSWORD={yourpassword} psql -h localhost -p 8000 -U postgres -d vulnmark -f cve_queries.sql



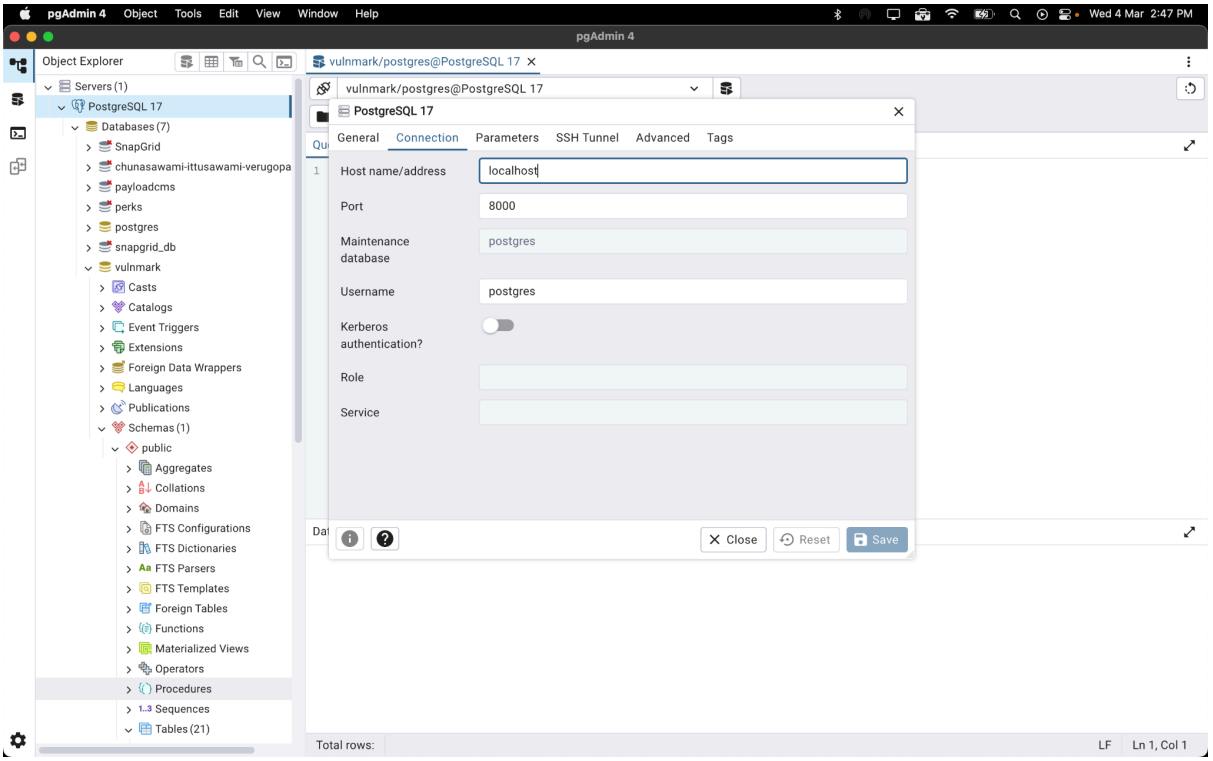


After the setup:

- Go to properties:



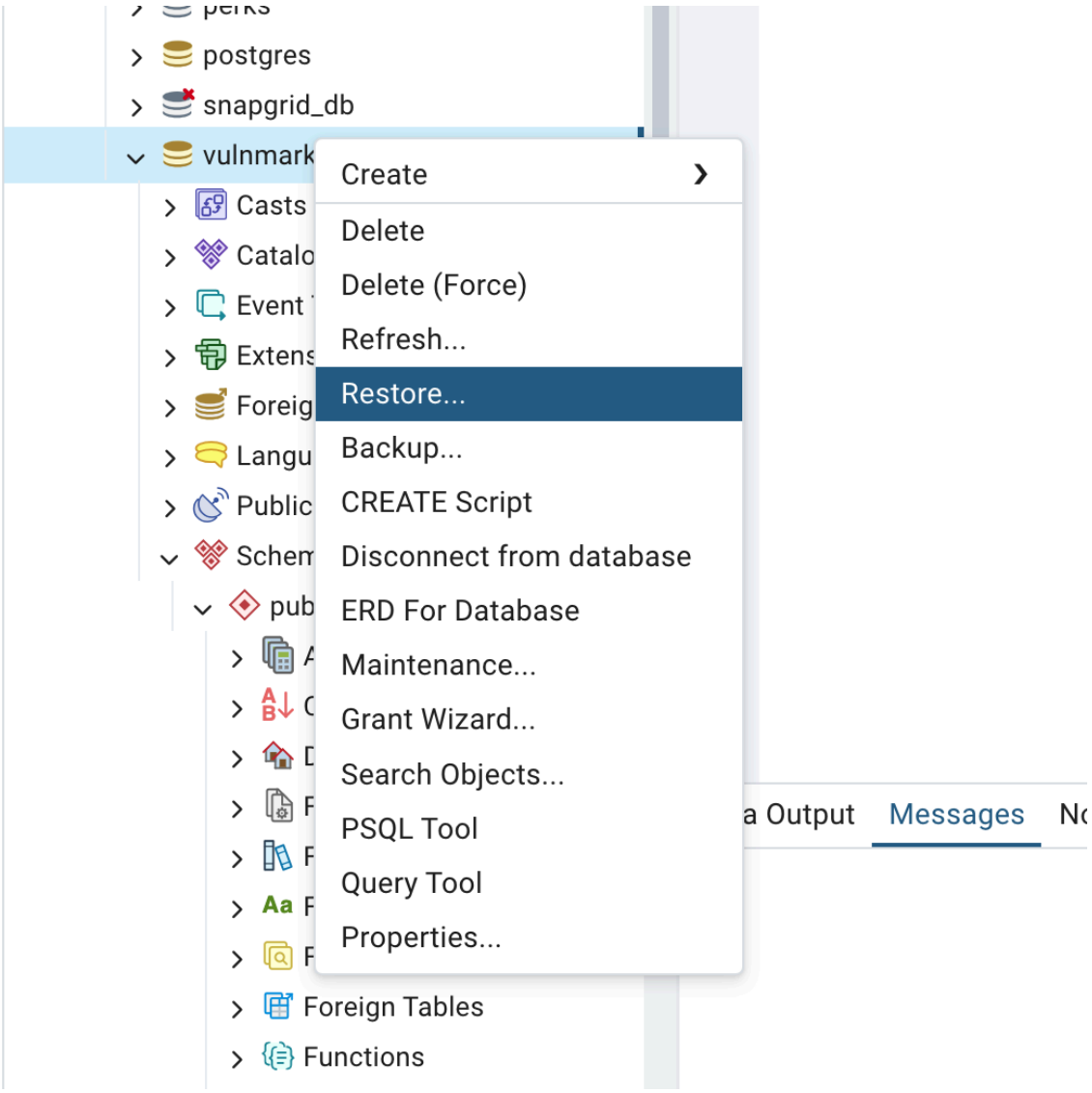
- **Copy these credentials and add to the env**

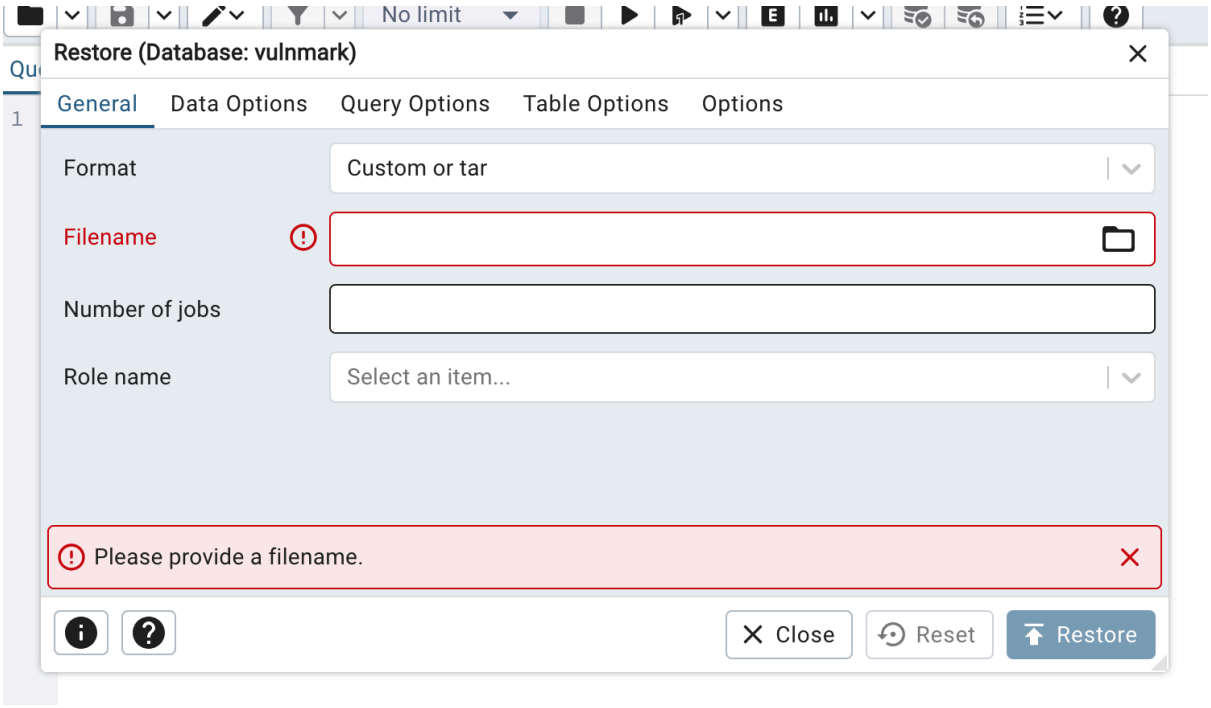


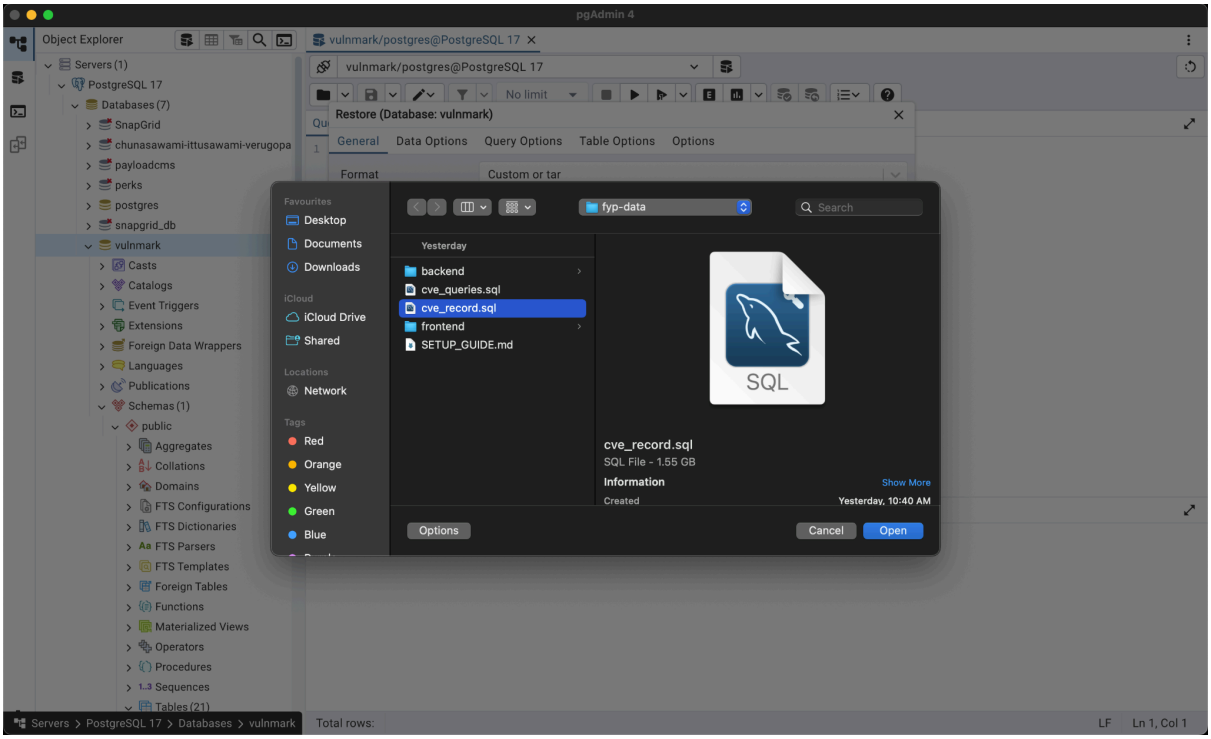
Step 5.3 — Download and restore core dump

1. Download dump from Drive and place it in project root.
2. Ensure dump filename is available (example: `cve_record.sql`).

Example:


postgres
snapgrid_db
vulnmark
Casts
Catalo
Event
Extens
Foreig
Langu
Public
Schem
pub
Foreign Tables
Functions
Create
Delete
Delete (Force)
Refresh...
Restore...
Backup...
CREATE Script
Disconnect from database
ERD For Database
Maintenance...
Grant Wizard...
Search Objects...
PSQL Tool
Query Tool
Properties...
a Output
Messages





If dump is PostgreSQL custom format, restore using `pg_restore`:

PGPASSWORD=mustafa23 pg_restore -h localhost -p 8000 -U postgres -d vulnmark --no-owner --no-privileges cve_record.sql

If `pg_restore` reports unsupported version:

- Install matching PostgreSQL client version (same major version as dump source), then retry.

Step 5.4 — Refresh materialized views (important)

After data import, run:

```
PGPASSWORD=mustafa23 psql -h localhost -p 8000 -U postgres -d vulnmark -c
"REFRESH MATERIALIZED VIEW severity_stats;"
PGPASSWORD=mustafa23 psql -h localhost -p 8000 -U postgres -d vulnmark -c
"REFRESH MATERIALIZED VIEW vendor_counts;"
PGPASSWORD=mustafa23 psql -h localhost -p 8000 -U postgres -d vulnmark -c
"REFRESH MATERIALIZED VIEW access_vector_counts;"
PGPASSWORD=mustafa23 psql -h localhost -p 8000 -U postgres -d vulnmark -c
"REFRESH MATERIALIZED VIEW category_type_counts;"
PGPASSWORD=mustafa23 psql -h localhost -p 8000 -U postgres -d vulnmark -c
"REFRESH MATERIALIZED VIEW cvss_score_ranges;"
PGPASSWORD=mustafa23 psql -h localhost -p 8000 -U postgres -d vulnmark -c
"REFRESH MATERIALIZED VIEW severity_probabilities;"
PGPASSWORD=mustafa23 psql -h localhost -p 8000 -U postgres -d vulnmark -c
"REFRESH MATERIALIZED VIEW epss_change_distribution;"
PGPASSWORD=mustafa23 psql -h localhost -p 8000 -U postgres -d vulnmark -c
"REFRESH MATERIALIZED VIEW cve_avg_percentages;"
PGPASSWORD=mustafa23 psql -h localhost -p 8000 -U postgres -d vulnmark -c
"REFRESH MATERIALIZED VIEW product_counts;"
PGPASSWORD=mustafa23 psql -h localhost -p 8000 -U postgres -d vulnmark -c
"REFRESH MATERIALIZED VIEW cve_counts_by_year_month;"
PGPASSWORD=mustafa23 psql -h localhost -p 8000 -U postgres -d vulnmark -c
"REFRESH MATERIALIZED VIEW cve_counts_by_year;"
PGPASSWORD=mustafa23 psql -h localhost -p 8000 -U postgres -d vulnmark -c
"REFRESH MATERIALIZED VIEW cve_counts_by_quarterly;"
PGPASSWORD=mustafa23 psql -h localhost -p 8000 -U postgres -d vulnmark -c
"REFRESH MATERIALIZED VIEW cve_counts_by_month_severity;"
PGPASSWORD=mustafa23 psql -h localhost -p 8000 -U postgres -d vulnmark -c
"REFRESH MATERIALIZED VIEW cve_counts_by_day_severity;"
PGPASSWORD=mustafa23 psql -h localhost -p 8000 -U postgres -d vulnmark -c
"REFRESH MATERIALIZED VIEW cve_data_view;"
```

---

6) Run backend server

From project root:

```
/absolute/path/to/.venv/bin/uvicorn backend.app.main:app --host 127.0.0.1 --port 8001
```

Or from inside backend:

uvicorn app.main:app --host 127.0.0.1 --port 8001

Backend Docs

curl http://127.0.0.1:8001/docs

Backend health test

curl http://127.0.0.1:8001/health

Expected:

{"status":"ok","service":"fastapi-cve-backend"}

---

7) Frontend setup

Step 7.1 — Install dependencies

cd frontend
bun install

Step 7.2 — Configure frontend env

Create frontend/.env.local:

LOCAL_API_BASE_URL=http://localhost:8001
NEXT_PUBLIC_LOCAL_API_BASE_URL=http://localhost:8001
NEXT_PUBLIC_SITE_URL=http://localhost:3000

(Template reference: frontend/.env.local.example)

Step 7.3 — Run frontend

bun run dev

Open: `http://localhost:3000`

---

8) Verification checklist

1. Backend health works:
    - `GET http://127.0.0.1:8001/health`
2. Dashboard endpoint returns data:
    - `GET http://127.0.0.1:8001/api/dashboard/cve-epss-ranges`
3. Frontend loads CVE pages without JSON parse errors.
4. CVE list/counts are not zero.

---

9) Common issues and fixes

Issue: `ModuleNotFoundError: No module named 'app'`

Cause: Running `uvicorn app.main:app` from project root.

Fix:

- From root use: `uvicorn backend.app.main:app ...`
- Or `cd backend` first, then `uvicorn app.main:app ...`

Issue: Backend returns `database_unavailable`

Cause: PostgreSQL not reachable or wrong `DATABASE_URL`.

Fix:

- Check DB host/port/user/password/dbname in backend/.env
- Confirm DB server is running

Issue: pgAdmin shows data but API shows zero

Cause: Backend points to different DB instance/port than pgAdmin.

Fix:

- Match `DATABASE_URL` exactly with pgAdmin server connection (host + port + dbname)

Issue: `pg_restore: unsupported version` for dump

Cause: Dump created from newer PostgreSQL version.

Fix:

- Install matching/newer `pg_restore`, then retry import.

---

10) Notes

You are only **allowed to** use the data and the source code for your Final Year Project. You're **not allowed** to **sell** or **share this data with anyone** else nor to **deploy the site with anyone**.

Good Luck and **Best Wishes** for your FYP.